Normalized vs Kimball data model

Vroomcheck · DAMG 6300 · what the two modelling styles are, and how they apply to this project

1 · In general: normalized (ER, 3NF) vs Kimball (dimensional)

	Normal (normalized / ER, 3NF)	Kimball (dimensional)
Built for	Running the business: inserting and updating records (OLTP)	Analysing the business: reading and summarising (OLAP)
Main goal	No duplicated data, so an update happens in one place	Fast, simple queries that people can understand
Shape	Many small tables linked in chains	A star: one central fact table with dimension tables around it
Starts from	The **entities** (vehicle, maker, component, owner…)	The **business process** (filing a complaint, issuing a recall…)
Tables	Entities and relationships, all treated the same	Two kinds: **facts** (events, with measures) and **dimensions** (descriptive context)
Redundancy	Removed (e.g. make is stored once in a `makes` table)	Allowed on purpose (make, model and model_year all sit in `dim_vehicle`)
Joins per question	Many, often 5 to 10	Few: the fact plus 1 to 3 dimensions
Key vocabulary	Primary keys, foreign keys, normal forms	Grain, measures (additive, semi-additive), conformed and role-playing dimensions, degenerate dimensions, bridges, SCD types

Example with Vroomcheck data: “deaths per model year for Ford”.

Normalized

complaints → vehicles → models → makes, with model year in its own table.
Four or more joins before you can add anything up.

Kimball

`fct_complaints` joined to `dim_vehicle`, `sum(deaths)`, group by `model_year`, filter `make = 'FORD'`.
One join.

2 · In this project: the two diagrams

> **The important point:** the tables are the same in both diagrams. Only the way of describing them differs. Vroomcheck's marts were designed the Kimball way from the start, so this is not two different databases.

Vroomcheck_Data_Model.png	Vroomcheck_Kimball_Data_Model.png
An **ER-style view**: tables, columns, keys and the lines between them	A **dimensional view**: the same tables labelled with Kimball's concepts
Answers “what tables and columns exist and how do they join?”	Answers “what business event does each table record, at what grain, and what can you add up?”
Shows PK and FK	Also shows **grain**, **degenerate dims** (odino, campno), **measure types** (additive vs semi-additive), **role-playing date** (one dim_date used four ways), **conformed dimension** (dim_vehicle shared by all four processes) and **SCD Type 1**
Good for developers	Good for explaining the design choices, which is what the presentation needs

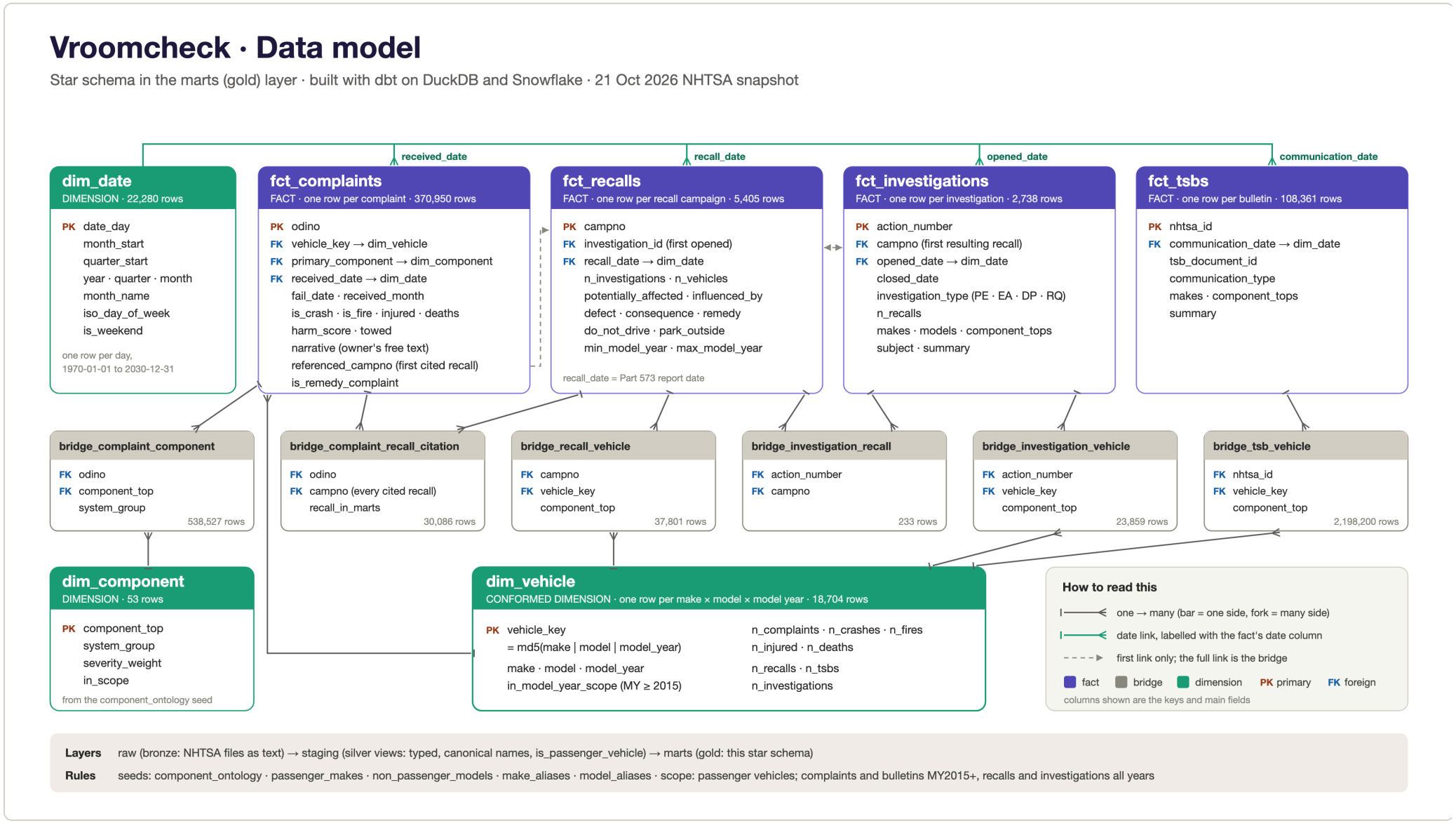
3 · One-line version for the presentation

> *“A normal data model is designed to store data without repeating it. A Kimball model is designed to answer questions about it. We start from the business events, declare the grain, and accept some duplication so every question is a simple join from a fact to its dimensions.”*

If someone asks whether you use normalization at all: yes, in a small way.

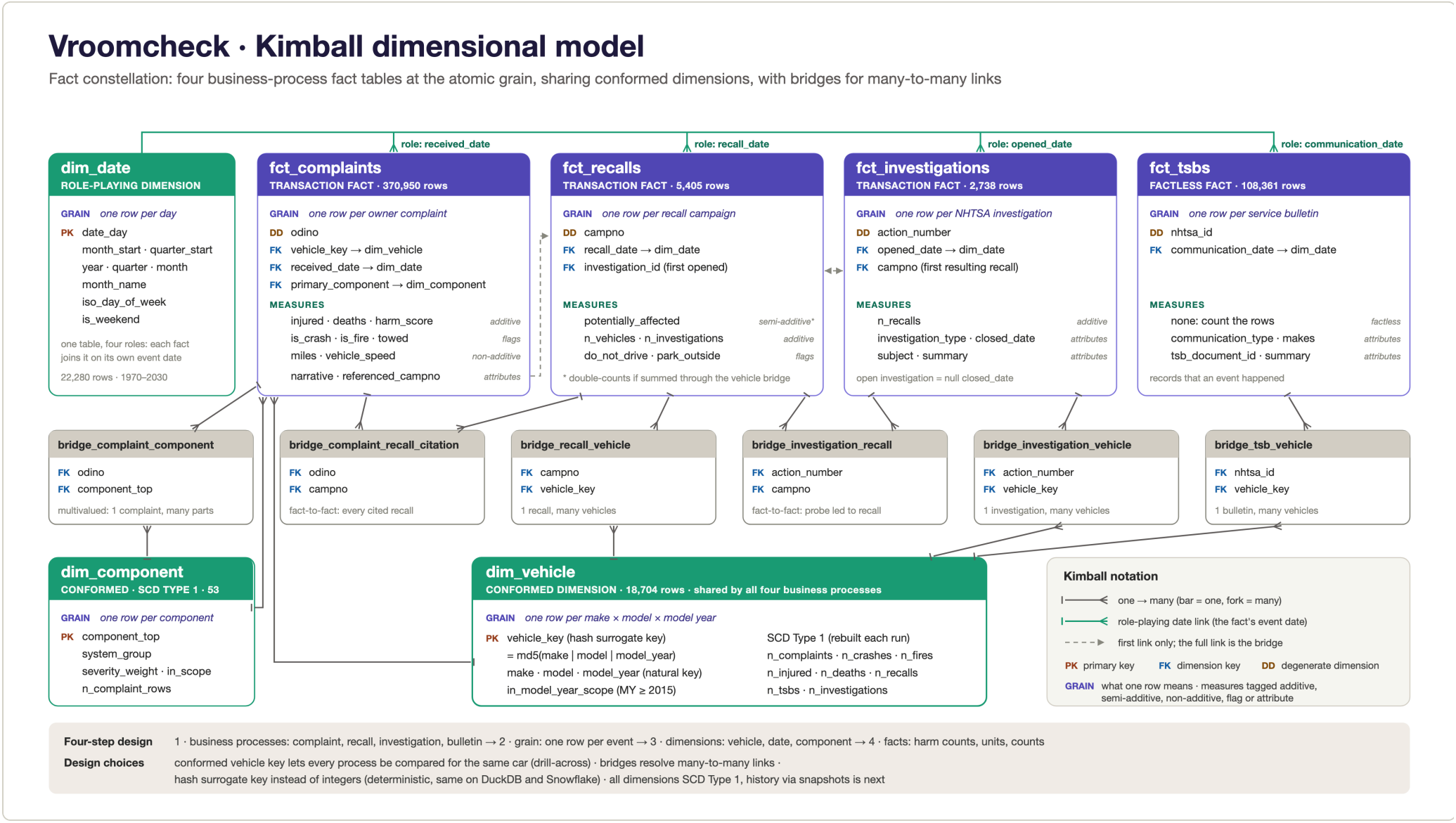
- The back room (raw, staging, ref seeds) is closer to source-system shape.
- The bridge tables are a normalized way to handle many-to-many links inside the dimensional model.
- The front room, the part users actually query, is pure Kimball.

Appendix A · ER-style view (Vroomcheck_Data_Model.png)



Tables, columns and join keys of the marts layer.

Appendix B · Kimball view (Vroomcheck_Kimball_Data_Model.png)



The same tables as a fact constellation: grain, measure types, conformed and role-playing dimensions, bridges.